

第4.2课 – 特殊类型与全局变量

深入理解Solidity中的地址类型、全局变量、枚举与常量



地址类型

address与address payable的区别及安全转账



全局变量

msg、block、tx对象的详细解析



枚举类型

状态机设计模式与应用场景



常量与不可变

Gas优化技巧与最佳实践

今天的学习内容

address类型

- 两种地址类型：address与address payable的区别
- 属性与方法：.balance、.code、.transfer()等
- 转账方法：transfer()、send()、call()的对比
- 安全实践：防止重入攻击的CEI模式

全局变量

- msg对象：sender、value、data、sig等
- block对象：timestamp、number、hash等
- tx对象：gasprice、origin等
- 全局函数：blockhash()、gasleft()等

枚举与常量

- enum枚举类型：定义有限的具名常量值
- 状态机模式：使用枚举管理合约生命周期
- constant常量：编译时确定的固定值
- immutable变量：部署时确定的不可变变量

众筹合约实战

- 综合应用：使用所有学过的知识点
- 安全考量：防止重入攻击等常见漏洞
- 部署测试：实际部署和测试智能合约

address类型 – 以太坊地址

基本定义

address类型是Solidity中专门用于存储以太坊账户地址的核心数据类型。

它可以存储两种类型的地址：

- 外部账户（EOA）地址
- 合约账户地址

 语法示例：

```
address public owner;
```

两种address类型

为了增强合约安全性，Solidity将地址类型细分为两种：

address

存储任何以太坊地址（EOA或合约地址）

address payable

存储可接收以太币的地址

以太坊地址格式

地址长度

20字节（160位）

表示方式

十六进制编码，以"0x"开头

示例

// 有效地址格式

0x742d35Cc6634C0532925a3b844Bc454e4438f44e

// 合约地址（部署后确定）

0x8ba1f109551bD43280301742D86f4bB191eA4441

两种address类型对比

特性

 **address**

 **address payable**

用途

 存储任何以太坊地址（EOA或合约地址）

 存储可接收以太币的地址（EOA或具备接收功能的合约地址）

接收以太币

 **不能**直接接收以太币

 **可以**接收以太币

转账方法

 不能使用.transfer()或.send()方法

 可以使用.transfer()和.send()方法

底层调用

 支持.call()、.delegatecall()、.staticcall()

 支持.call()、.delegatecall()、.staticcall()

安全性

 编译器在编译时检查，防止误转账

 需显式声明，明确表示该地址可接收以太币

类型转换

 可隐式转换为address

 需显式使用payable(addr)转换为address payable

 payable是payable的，不是address的

address的属性和方法

属性

.balance

uint

查询地址的以太币余额，单位为wei（1 ether = 10¹⁸ wei）

```
uint balance = address(recipient).balance;
```

.code

bytes memory

获取地址的合约代码。对于外部账户(EOA)，此属性为空；对于合约账户，此属性为非空

```
bytes code = address(recipient).code;
```

.codehash

bytes32

获取地址的合约代码的Keccak-256哈希值

```
bytes32 hash = address(recipient).codehash;
```

方法

.transfer()

address payable

向指定地址转账amount(wei)。如果转账失败，交易将自动回退(revert)。Gas限制为2300

```
recipient.transfer(amount);
```

.send()

address payable

与transfer类似，但失败时不会自动回退，而是返回一个布尔值表示成功或失败

```
bool success = recipient.send(amount);
```

.call()

所有address

通用底层调用方法。可以用于调用其他合约的函数，或向地址转账。返回布尔值表示调用是否成功

```
bool success; (success, bytes memory data) = recipient.call{value: amount}("");
```

转账方法对比

↔ transfer()

语法: recipient.transfer(amount)

Gas限制: 2300

失败处理: 自动回退(revert)

安全性: 相对安全, 失败即回退

推荐度: 传统安全方案

➡ send()

语法: bool success = recipient.send(amount)

Gas限制: 2300

失败处理: 返回false, 不自动回退

安全性: 易出错, 可能导致资金丢失

推荐度: 不推荐 (易出错)

🔧 call()

语法: (bool success,) = recipient.call{value: amount}("")

Gas限制: 无固定限制, 可自定义

失败处理: 返回false, 不自动回退

安全性: 最灵活, 但存在重入风险

推荐度: 推荐 (2024年后)

特性	transfer()	send()	call()
使用场景	简单的以太坊转账	不推荐使用	复杂调用, 需要更多Gas
Gas消耗	固定2300 Gas	固定2300 Gas	可自定义, 更灵活
返回值	无返回值	bool类型	(bool, bytes)

</> 代码示例:

<pre>transfer(): recipient.transfer(amount);</pre>	<pre>send(): bool success = recipient.send(amount);</pre>	<pre>call(): (bool success,) = recipient.call{value: amount}("");</pre>
--	---	--

转账安全

⚠️ 重入攻击风险

在外部调用后更新状态，会导致重入攻击：

```
function withdraw() public {
    uint amount = balances[msg.sender];

    // 危险：先转账
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent, "Failed");

    // 后更新状态（太晚了！）
    balances[msg.sender] = 0;
}
```

攻击流程：

- 攻击者调用`withdraw()`
- 合约转账给攻击者
- 在`fallback`函数中再次调用`withdraw()`
- 由于 `balances` 尚未更新，攻击者可重复提取资金

✅ CEI模式 (Checks–Effects–Interactions)

智能合约安全的黄金法则，防止重入攻击：

Checks

先检查
执行前验证

Effects

后更新
修改状态

Interactions

最后调用
外部交互

```
function withdraw() public {
    uint amount = balances[msg.sender];

    // 安全：先更新状态
    balances[msg.sender] = 0;

    // 后转账
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent, "Transfer failed");
}
```

关键点：

- 总是先更新状态，再进行外部调用
- 使用`require`检查转账结果
- 对可疑的外部调用使用`call`并手动检查返回值

全局变量

Solidity提供的一组内置特殊变量，允许智能合约访问关于区块链、交易和当前调用上下文的丰富信息

msg对象

包含当前函数调用的相关信息

- `sender`: 直接调用者地址
- `value`: 随调用发送的以太币数量
- `data`: 完整的调用数据

block对象

提供当前区块的链上环境信息

- `timestamp`: 区块时间戳
- `number`: 区块号
- `hash`: 区块哈希值

tx对象

包含交易相关的信息

- `gasprice`: 交易Gas价格
- `origin`: 交易发起者地址

重要全局函数

Solidity提供的实用函数

- `blockhash()`: 获取区块哈希
- `gasleft()`: 剩余Gas
- `selfdestruct()`: 自毁合约

 全局变量是Solidity的关键特性，正确使用它们可以简化开发并提高合约安全性

msg对象 – 调用信息

msg.sender

类型 address

当前函数的直接调用者

用途

- 权限控制：require(msg.sender == owner, "Not owner");
- 记录操作者：在事件或状态变量中记录执行操作的地址
- 身份验证：作为用户身份的标识

```
require(msg.sender == owner, "Not the owner");
balances[msg.sender] += amount;
```

msg.value

类型 uint (单位：wei)

随当前调用发送的以太币数量

用途

- 支付功能：实现购买、捐赠等需要接收以太币的逻辑
- 众筹/捐款：记录用户贡献的金额
- 购买NFT：验证支付金额是否正确

```
require(msg.value == price, "Incorrect payment");
require(msg.value >= minAmount, "Too low");
```

msg.data

类型 bytes calldata

完整的调用数据，包括函数选择器和所有编码后的参数

用途

- 代理合约：将原始调用数据转发给其他合约
- 元交易：在某些高级场景中处理原始交易数据
- 调试：分析合约调用时的原始输入

```
(bool success, ) = target.call(msg.data);
```

msg.sig

类型 bytes4

函数选择器，即msg.data的前4个字节

用途

- 函数路由：在fallback或receive函数中识别被调用的函数
- 权限检查：根据函数选择器进行特定的权限判断
- 事件记录：记录被调用的函数标识符

```
bytes4 selector = msg.sig;
```

block对象 – 区块信息

block.timestamp

类型：uint (Unix时间戳)

说明：当前区块的Unix时间戳

用途：时间锁、截止日期、时间间隔限制



注意：矿工可在约15秒范围内操纵此值，**不要用于生成关键随机数**

block.number

类型：uint (区块号)

说明：当前区块的区块号 (高度)

用途：计算区块数、快照功能、基于区块的锁定



以太坊主网：平均每12-14秒产生一个区块，每天约6400个区块

blockhash(uint blockNumber)

类型：bytes32

说明：获取指定区块的哈希值

限制：只能获取最近256个区块的哈希



注意：不要用于重要的随机性生成，推荐使用Chainlink VRF

其他block属性

block.gaslimit：当前区块的Gas限制

block.coinbase：挖出当前区块的矿工地址

block.difficulty：当前区块的难度值 (现为block.prevrandao的别名)

 时间单位： 1 seconds = 1 | 1 minutes = 60 | 1 hours = 3600 | 1 days = 86400 | 1 weeks = 604800

tx对象与安全

tx.origin

类型：address

含义：交易的原始发送者（EOA）

安全风险

在权限检查中使用tx.origin可能导致严重的安全隐患

msg.sender vs tx.origin

特性	msg.sender	tx.origin
调用者	直接调用者	原始发送者
合约调用	安全	危险

危险示例

```
function transferOwnership(address newOwner) {
    require(tx.origin == owner, "Not owner");
    // ...
}
```

安全建议

```
function transferOwnership(address newOwner) {
    require(msg.sender == owner, "Not owner");
    // ...
}
```

重要的全局函数

gasleft()

功能：返回当前交易剩余的Gas数量

语法： `function gasleft() internal pure returns (uint256)`

应用场景：

- Gas优化：动态调整执行路径
- 防止Gas不足导致的交易失败
- 实现更精细的Gas消耗控制

keccak256()

功能：执行Keccak-256哈希函数

语法： `function keccak256(bytes memory) internal pure returns (bytes32)`

应用场景：

- 创建唯一标识符
- 密码哈希存储
- 生成随机数（配合时间戳）
- 确定性交易哈希计算

abi.encode()

功能：执行ABI编码

语法： `function abi.encode(...) internal pure returns (bytes memory)`

应用场景：

- 构建结构化数据
- 准备函数调用数据
- 创建复合哈希
- 与低级调用一起使用

☰ 其他全局函数

`blockhash(uint)`

获取指定区块的哈希值（仅最近256个区块）

`tx.origin`

交易发起者地址（慎用，可能存在安全风险）

`msg.sender`

消息发送者地址（当前调用者）

enum – 枚举类型概述

定义与特点

- 枚举（`enum`）是Solidity中一种用户自定义的数据类型
- 用于定义一组有限的、具名的常量值
- 这些常量值在代码中代表特定的意义，且取值范围固定明确
- 底层实现上是`uint8`类型，从0开始按顺序编号

定义与使用示例：

```
// 定义枚举
enum Status {
    Pending,      // 0
    Active,       // 1
    Completed,    // 2
    Cancelled     // 3
}

// 声明枚举变量
Status public currentStatus; // 默认值：Pending (0)

// 设置值
currentStatus = Status.Active;

// 检查值
if (currentStatus == Status.Completed) {
    // 执行逻辑...
    currentStatus = Status.Cancelled;
}
```

枚举类型的优势

代码可读性增强

使用具名常量（如`Status.Completed`）比使用数字（如`2`）更直观，使代码意图一目了然

类型安全

编译器会在编译时检查枚举值的有效性，防止赋给无效值，从而减少运行时错误

节省Gas

相比于使用`string`类型来表示状态，枚举类型在存储和操作上更高效，从而节省Gas费用

🔄 状态机设计模式

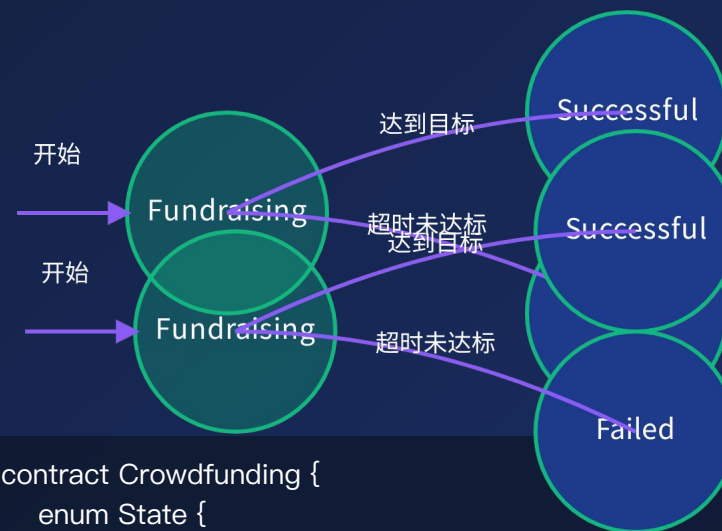
⚙️ 状态机概念

状态机设计模式通过枚举类型来管理合约的生命周期或流程状态，确保操作在正确的状态下执行。

</> 众筹合约状态转换

- 初始状态: Fundraising (募资中)
- 达到目标: 转换为 Successful (成功)
- 超时未达标: 转换为 Failed (失败)
- 状态转换受**修饰符**保护，确保安全性

```
modifier inState(State expectedState) {  
    require(currentState == expectedState, "Invalid state");  
    _;  
}
```



```
contract Crowdfunding {  
    enum State {  
        Fundraising, // 募资中  
        Successful, // 成功  
        Failed,      // 失败  
        Closed       // 已关闭  
    }  
  
    State public currentState;  
  
    // 修饰符：确保函数在特定状态下执行  
    modifier inState(State _state) {  
        require(currentState == _state, "Invalid state");  
        _;  
    }  
  
    // 只有在募资中才能贡献  
    function contribute()
```

常量与不可变变量

constant 与 immutable 的特性对比与应用场景

特性	constant	immutable
赋值时机	编译时确定，声明时必须赋值	部署时（在构造函数中）赋值一次
存储位置	不占用合约存储空间，直接嵌入到字节码中	不占用合约存储空间，存储在合约代码中
Gas成本	读取成本最低，因为值直接硬编码	读取成本低，但略高于constant，因为需要在部署时读取
用途	编译时已知且永不改变的固定值（如数学常数、固定地址）	需要在运行时（如构造函数中）计算或确定的值（如合约部署者地址）

</> constant 示例

```
uint256 public constant PI = 314159; // 编译时确定的值
address public constant OWNER = 0x1234...; // 固定地址
string public constant NAME = "MyToken"; // 字符串常量

// 读取常量
function getPi() public pure returns (uint256) {
    return PI; // 直接嵌入到字节码中，Gas成本最低
}
```

⚙ immutable 示例

```
address public immutable owner; // 声明immutable变量
uint256 public immutable deadline;

constructor() {
    owner = msg.sender; // 在构造函数中赋值
    deadline = block.timestamp + 7 days;
}

function checkDeadline() public view {
    require(block.timestamp < deadline); // 稍高Gas成本，但更灵活
}
```

安全与Gas优化总结



address类型安全

- ✓ 明确区分address和address payable类型
- ✓ 优先使用call方法进行转账，并检查返回值
- ⚠ 遵循CEI模式(Checks-Effects-Interactions)防止重入攻击
- ✓ 先更新状态，后进行外部调用
- ✓ 避免使用transfer和send而不检查返回值



全局变量使用规范

- ✓ 使用msg.sender进行权限控制和身份验证
- ✓ 验证msg.value以确保正确支付
- ⚠ 避免使用block.timestamp生成随机数
- ✓ 合理使用blockhash验证历史数据
- ✓ 了解msg.data和msg.sig的用途



Gas优化实用技巧

- ✓ 对编译时已知的值使用constant
- ✓ 对部署时确定的值使用immutable
- ✓ 使用enum类型管理状态，节省存储空间
- ✓ 设计高效的事件用于链下应用
- ✓ 使用modifier简化重复代码



最佳实践: 安全性与Gas优化同等重要，优先确保合约安全，再考虑优化Gas成本。在众筹合约中，同时应用了这些安全和优化技巧。

众筹合约完整实战（上）

核心结构设计与状态管理

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract Crowdfunding {
    // 枚举定义众筹项目的状态
    enum State {
        Fundraising,    // 募资中
        Succeeded,      // 成功
        Failed,          // 失败
        Withdrawn        // 已提取
    }
}
```

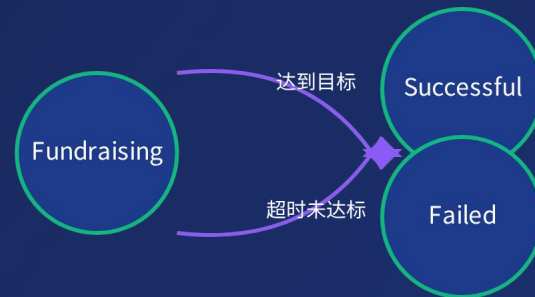
```
// immutable变量：部署时确定，不可更改，节省Gas
address public immutable owner;    // 项目发起人
uint256 public immutable goalAmount; // 筹款目标金额 (wei)
uint256 public immutable deadline;  // 众筹截止时间 (Unix timestamp)
```

```
// constant变量：编译时确定，不可更改，节省Gas
uint256 public constant MIN_CONTRIBUTION = 0.01 ether; // 最小贡献金额
```

```
// 状态变量
State public currentState;    // 当前众筹状态
uint256 public currentRaisedAmount; // 当前已筹集金额
mapping(address => uint256) public contributions; // 记录每个地址的贡献金额
```

```
// 事件：用于链下应用监听合约状态变化
event ContributionReceived(address indexed contributor, uint256 amount);
```

众筹状态转换图



核心设计要点

- ✓ **immutable关键字**：确保projectOwner和fundingGoal部署后不可更改，提高合约安全性
- ✓ **enum枚举类型**：State枚举清晰定义了众筹生命周期状态，确保操作在正确状态下执行
- ✓ **修饰符**：inState确保函数只在特定状态下执行，简化了状态检查
- ✓ **状态转换逻辑**：在contribute和endCampaign函数中实现，根据条件更新currentState
- ✓ **双重检查机制**：在endCampaign中再次验证状态，防止重复操作

众筹合约完整实战（下）

资金提取功能

```
// 提取资金函数：项目成功后，发起人可以提取资金
function withdrawFunds() public onlyOwner inState(State.成功) {
    // CEI模式：Effects (更新内部状态)
    currentState = State.已提取; // 状态变为已提取，防止重复提取
    emit StateChanged(currentState);

    uint256 amount = address(this).balance;

    // Interactions (与外部交互)
    // 使用call方法进行转账，并检查返回值，防止重入攻击
    (bool success, ) = payable(owner).call{value: amount}("");
    require(success, "Failed to withdraw funds");

    emit FundsWithdrawn(owner, amount);
}; }
```

- 权限控制：仅项目方可提取资金
- 状态检查：确保众筹已成功且结束
- CEI模式：先更新状态，后执行外部调用
- 双重检查：确保达到目标金额

退款功能

```
// 退款函数：项目失败后，贡献者可以申请退款
function refund() public {
    _checkState(); // 确保状态已更新为失败

    require(currentState == State.失败, "Project is not in failed state for refund");
    uint256 amountToRefund = contributions[msg.sender];
    require(amountToRefund > 0, "No contribution to refund");

    // CEI模式：Effects (更新内部状态)
    contributions[msg.sender] = 0; // 清零贡献记录，防止重入攻击

    // Interactions (与外部交互)
    // 使用call方法进行转账，并检查返回值
    (bool success, ) = payable(msg.sender).call{value: amountToRefund}("");
    require(success, "Failed to refund");

    emit RefundIssued(msg.sender, amountToRefund);
}
```

- 状态检查：仅在众筹失败时可用
- 余额检查：确保有可退还的金额
- CEI模式：先更新贡献记录，后执行转账
- 事件emit：记录退款事件便于链下监控

安全最佳实践

CEI模式

返回值检查

状态验证

合约部署和测试演示

部署流程

- 1 编译合约: solc compiler
- 2 部署脚本: 编写部署脚本
- 3 部署到测试网: ganache或hardhat

```
const Crowdfunding = artifacts.require("Crowdfunding");  
const deployed = await Crowdfunding.new(goal, duration);
```

功能测试

- 1 募资测试: contribute()函数
- 2 成功场景: 达到目标金额
- 3 失败场景: 未达目标金额

```
// ...  
await deployed.contribute({value: web3.utils.toWei("1", "ether")});  
const balance = await deployed.getContractBalance();  
// ...
```

关键场景验证

- 1 时间限制: 测试截止时间
- 2 重入攻击: 模拟攻击测试
- 3 状态转换: 验证状态机正确性

```
// ...  
await deployed.endCampaign();  
const state = await deployed.currentState(); // 状态机验证  
// ...
```

课程总结与作业布置

✔ 核心知识点

- **address类型**: 值检查、转账方法、安全模式
- **全局变量**: msg、block、tx对象的使用
- **枚举类型**: 状态机设计模式的应用
- **常量与不可变**: Gas优化技巧

☞ 实战作业

自己实现一个众筹合约:

- 🔗 添加管理员权限控制
- 🔗 实现部分退款功能
- 🔗 使用事件记录关键操作
- 🔗 添加更多安全检查

提交地址: github.com/yourname/advanced-crowdfunding